TITAN OMNI: COMPLETE VOICE-CONTROLLED BUSINESS OPERATING SYSTEM
Universal AI-Powered Business Command Center - Every Channel, Every Device, Every Action
System: Titan Omni (Part of TitanBOS - Titan Business Operating System)AI Core: Titan Zero (Six-core reasoning pipeline - LogiCore, CreatiCore, OmegaCore, OmicronCore, EntropyCore, EquilibriumCore)Timeline: 32 weeks (8 months)Team: 8 developers + 1 full-stack architect + 1 voice/AI specialist + 1 DevOps + 1 designer + 2 QA + 1 product architectTotal LOC: 80,000-100,000 lines addedPlatforms: SMS, WhatsApp, Telegram, Messenger, Slack, Custom APIs, PWA (all devices), iOS, Android, Wearables, Tablets, Desktop, WebTarget Users: Everyone - voice is primary interface, visual UI optional fallback

CORE CONCEPT: VOICE IS THE PRIMARY UI

TITAN OMNI PHILOSOPHY:

Traditional App: "Open app → Navigate menus → Find feature → Click → Confirm"
TITAN OMNI: "Say what you want → AI understands → Action happens"

TRADITIONAL WORKFLOW: "I need to book a service"
→ Open TitanPortal
→ Click "Book Service"
→ Fill form (customer, date, location)
→ Review
→ Confirm
→ Wait for sync
Time: 3-5 minutes

TITAN OMNI WORKFLOW: "Book cleaning service for John at 456 Main St next Tuesday"
→ OMNI understands
→ Validates with Titan Zero
→ Creates service request
→ Confirms via voice
Time: 20 seconds

KEY PRINCIPLE:
Every action in TitanBOS is voice-executable through Titan Omni.
No mouse required. No menu navigation. No forms. Just voice.
Visual UI exists only as accessibility fallback.

PHASE 0: FOUNDATION (Weeks 1-4)
Unified Command Interface, Voice I/O, Channel Architecture, Titan Zero Integration
0.1 Universal Voice Command Architecture (16 hours)

```
//
================================================================================
// TITAN OMNI: UNIVERSAL VOICE COMMAND INTERFACE
// Complete Business OS Voice Control
//
================================================================================

// Voice command context - understands what user wants to do
class VoiceCommand {
  final String rawInput;           // User's exact words
  final String intent;             // What they want: "book_service", "check_status"
  final String action;             // What to execute: "create_job", "fetch_invoice"
  final Map<String, dynamic> params; // Extracted data
  final double confidence;         // How sure Titan Zero is (0.0-1.0)

  // Titan Zero reasoning
  final ValidationResult logicValidation;    // LogiCore: Is this allowed?
  final OptimizationResult creatiOptimization; // CreatiCore: How to execute?
  final InsightResult omegaInsight;         // OmegaCore: What should happen?
  final OmicronResult odicronAnalysis;      // OmicronCore: Context analysis
  final EntropyResult entropyProcessing;    // EntropyCore: Chaos/edge cases
  final EquilibriumResult equilibriumBalance; // EquilibriumCore: Optimal outcome

  // Channel & device
  final String channel;            // "sms", "whatsapp", "web", "mobile_app", "wearable"
  final String deviceId;
  final String? audioUrl;          // User's voice recording
  final String? transcript;        // Transcribed text

  const VoiceCommand({
    required this.rawInput,
    required this.intent,
    required this.action,
    required this.params,
    required this.confidence,
    required this.logicValidation,
    required this.creatiOptimization,
    required this.omegaInsight,
    required this.odicronAnalysis,
    required this.entropyProcessing,
    required this.equilibriumBalance,
    required this.channel,
```

```
    required this.deviceId,
    this.audioUrl,
    this.transcript,
  });
}

// Every business action is a voice command
class VoiceCommandRegistry {
  static const commands = {
    // JOB MANAGEMENT
    'book_service': {
      'action': 'create_job',
      'params': ['service_type', 'customer', 'date', 'location'],
      'required': ['service_type'],
      'examples': [
        'Book cleaning service for John Smith next Tuesday',
        'Schedule plumbing at 123 Main Street',
        'Create job: electrical work, customer Mike, tomorrow 10am',
      ],
    },

    'check_job_status': {
      'action': 'fetch_job_status',
      'params': ['job_id', 'customer_name'],
      'required': ['job_id'],
      'examples': [
        'What\'s the status of job 12345?',
        'Where is John\'s service?',
        'Check my pending jobs',
      ],
    },

    'assign_job': {
      'action': 'assign_job',
      'params': ['job_id', 'technician_name'],
      'required': ['job_id', 'technician_name'],
      'examples': [
        'Assign job 12345 to Mike',
        'Give the plumbing job to Sarah',
      ],
    },

    'complete_job': {
      'action': 'update_job_status',
```

```
  'params': ['job_id', 'status'],
  'required': ['job_id'],
  'examples': [
    'Mark job 12345 as complete',
    'Finish the cleaning service',
  ],
},

// CUSTOMER MANAGEMENT
'add_customer': {
  'action': 'create_customer',
  'params': ['name', 'phone', 'email', 'address'],
  'required': ['name'],
  'examples': [
    'Add customer John Smith, phone 0412345678, email john@example.com',
    'New customer: Sarah, 123 Main Street',
  ],
},

'get_customer_info': {
  'action': 'fetch_customer',
  'params': ['customer_name'],
  'required': ['customer_name'],
  'examples': [
    'Show me John Smith\'s details',
    'What\'s Sarah\'s phone number?',
  ],
},

// INVOICING & PAYMENT
'create_invoice': {
  'action': 'create_invoice',
  'params': ['customer', 'amount', 'description', 'job_id'],
  'required': ['customer', 'amount'],
  'examples': [
    'Invoice John for 250 dollars for plumbing',
    'Create invoice: Sarah, 500 dollars, cleaning service',
  ],
},

'send_invoice': {
  'action': 'send_invoice',
  'params': ['invoice_id', 'customer_email'],
  'required': ['invoice_id'],
```

```
  'examples': [
    'Send invoice 123 to John',
    'Email the latest invoice to Sarah',
  ],
},

'check_payment': {
  'action': 'fetch_invoice',
  'params': ['invoice_id', 'customer_name'],
  'examples': [
    'Has John paid the invoice?',
    'Check payment status for invoice 123',
  ],
},

'record_payment': {
  'action': 'record_payment',
  'params': ['invoice_id', 'amount', 'method'],
  'required': ['invoice_id'],
  'examples': [
    'Record payment of 250 dollars for invoice 123',
    'John paid in cash, update invoice',
  ],
},

// TEAM & SCHEDULING
'assign_technician': {
  'action': 'assign_technician',
  'params': ['technician_name', 'date', 'job_count'],
  'required': ['technician_name', 'date'],
  'examples': [
    'Assign Mike 3 jobs for Monday',
    'Schedule Sarah on Wednesday',
  ],
},

'check_availability': {
  'action': 'fetch_availability',
  'params': ['technician_name', 'date'],
  'required': ['technician_name'],
  'examples': [
    'Is Mike available Thursday?',
    'Show Sarah\'s schedule',
  ],
```

```
},

// ANALYTICS & REPORTING
'business_summary': {
  'action': 'fetch_dashboard',
  'params': [],
  'examples': [
    'How\'s business today?',
    'What\'s my daily summary?',
    'Revenue report',
  ],
},

'monthly_revenue': {
  'action': 'fetch_revenue_report',
  'params': ['month'],
  'examples': [
    'What did I make in March?',
    'Monthly revenue report',
  ],
},

'outstanding_invoices': {
  'action': 'fetch_outstanding',
  'params': [],
  'examples': [
    'Who owes me money?',
    'Show unpaid invoices',
  ],
},

// COMPLIANCE & SAFETY
'safety_check': {
  'action': 'fetch_compliance_status',
  'params': [],
  'examples': [
    'Are my permits current?',
    'Compliance status',
  ],
},

// TEAM COMMUNICATION
'message_team': {
  'action': 'send_team_message',
```

```
      'params': ['technician', 'message'],
      'required': ['technician', 'message'],
      'examples': [
        'Tell Mike the customer changed the address',
        'Message Sarah: John\'s not home, reschedule',
      ],
    },

    // HELP & DOCUMENTATION
    'help': {
      'action': 'show_help',
      'params': ['topic'],
      'examples': [
        'How do I book a service?',
        'Help with invoicing',
        'What can I do with Omni?',
      ],
    },
  };
}

// Universal voice input - works on all channels
class UniversalVoiceInput extends GetxService {
  final voiceEngine = Get.find<VoiceEngine>();
  final titanZero = Get.find<TitanZeroRequestRouter>();

  final isListening = false.obs;
  final currentCommand = Rxn<VoiceCommand>();
  final commandQueue = <VoiceCommand>[].obs;

  // Listen for voice commands on any channel
  Future<VoiceCommand?> listenForCommand({
    required String channel,
    int timeoutSeconds = 30,
    String? context, // Optional context: "booking" vs "payment"
  }) async {
    try {
      isListening.value = true;

      // Start listening on channel
      final input = await voiceEngine.startListening(
        timeoutSeconds: timeoutSeconds,
        showUI: false,
      );
```

```
    if (input == null) {
      isListening.value = false;
      return null;
    }

    // Send to Titan Zero for understanding
    final command = await _processWithTitanZero(
      input: input.text,
      transcript: input.audioTranscript,
      audioUrl: input.audioUrl,
      channel: channel,
      context: context,
    );

    isListening.value = false;

    if (command != null) {
      currentCommand.value = command;
      commandQueue.add(command);
    }

    return command;
  } catch (e) {
    print('Error listening for command: $e');
    isListening.value = false;
    return null;
  }
}

// Process command through Titan Zero reasoning pipeline
Future<VoiceCommand?> _processWithTitanZero({
  required String input,
  String? transcript,
  String? audioUrl,
  required String channel,
  String? context,
}) async {
  try {
    // 1. LOGICORE: Validate intent is allowed
    final intentRecognition = await titanZero.logicCore.recognizeIntent(
      input: input,
      context: context,
    );
```

```
    if (!intentRecognition.isValid) {
      // Intent not recognized or not allowed
      await voiceEngine.speak(
        'I didn\'t understand that. ${intentRecognition.suggestion ?? "Try saying: book a service,
check status, or create an invoice."}'
      );
      return null;
    }

    final intent = intentRecognition.intent;

    // 2. Extract parameters from input
    final params = await titanZero.logicCore.extractParameters(
      input: input,
      intent: intent,
    );

    // 3. Check if all required parameters are present
    final requiredParams = VoiceCommandRegistry.commands[intent]?['required'] ?? [];
    final missingParams = requiredParams.where((p) => !params.containsKey(p)).toList();

    if (missingParams.isNotEmpty) {
      // Ask for missing information
      final question = await titanZero.creatiCore.generateClarificationQuestion(
        intent: intent,
        missingParams: missingParams,
      );

      await voiceEngine.speak(question);

      // Listen for clarification
      final clarification = await listenForCommand(
        channel: channel,
        context: intent,
      );

      if (clarification != null) {
        // Merge params
        clarification.params.forEach((key, value) {
          params[key] = value;
        });
      }
    }
```

```
// 4. LOGICORE: Validate the command
final validation = await titanZero.logicCore.validateCommand(
  intent: intent,
  params: params,
);

if (!validation.isValid) {
  await voiceEngine.speak(
    'I can\'t do that. ${validation.reason ?? "Please contact support."}'
  );
  return null;
}

// 5. CREATICORE: Optimize how to execute
final optimization = await titanZero.creatiCore.optimizeExecution(
  intent: intent,
  params: params,
);

// 6. OMEGACORE: Gather context and insights
final insight = await titanZero.omegaCore.analyzeCommand(
  intent: intent,
  params: params,
);

// 7. OMIC RONCORE: Analyze command in business context
final analysis = await titanZero.omicronCore.analyzeContext(
  intent: intent,
  params: params,
);

// 8. ENTROPYCORE: Handle edge cases, chaos, exceptions
final entropy = await titanZero.entropyCore.handleEdgeCases(
  intent: intent,
  params: params,
  validation: validation,
);

// 9. EQUILIBRIUM CORE: Balance competing goals, optimize outcomes
final equilibrium = await titanZero.equilibriumCore.balanceOutcomes(
  intent: intent,
  params: params,
  optimization: optimization,
```

```
      insight: insight,
    );

    // 10. Construct final command
    final command = VoiceCommand(
      rawInput: input,
      intent: intent,
      action: VoiceCommandRegistry.commands[intent]?['action'] ?? intent,
      params: params,
      confidence: intentRecognition.confidence,
      logicValidation: validation,
      creatiOptimization: optimization,
      omegaInsight: insight,
      odicronAnalysis: analysis,
      entropyProcessing: entropy,
      equilibriumBalance: equilibrium,
      channel: channel,
      deviceId: getCurrentDeviceId(),
      audioUrl: audioUrl,
      transcript: transcript,
    );

    return command;
  } catch (e) {
    print('Error processing command: $e');
    return null;
  }
}

// Execute the command
Future<CommandResult> executeCommand(VoiceCommand command) async {
  try {
    // Route to appropriate handler based on action
    final handler = _getActionHandler(command.action);

    if (handler == null) {
      return CommandResult(
        success: false,
        error: 'Unknown action: ${command.action}',
      );
    }

    // Execute with authorization checks
    final result = await handler.execute(command);
```

```
    // Provide voice feedback
    await voiceEngine.speak(result.voiceMessage);

    // Log command execution
    await _logCommand(command, result);

    return result;
  } catch (e) {
    print('Error executing command: $e');

    return CommandResult(
      success: false,
      error: e.toString(),
      voiceMessage: 'An error occurred: ${e.toString()}',
    );
  }
}

ActionHandler? _getActionHandler(String action) {
  switch (action) {
    case 'create_job':
      return JobActionHandler();
    case 'fetch_job_status':
      return JobStatusHandler();
    case 'create_customer':
      return CustomerActionHandler();
    case 'create_invoice':
      return InvoiceActionHandler();
    case 'fetch_dashboard':
      return DashboardHandler();
    // ... add all handlers
    default:
      return null;
  }
}

Future<void> _logCommand(VoiceCommand command, CommandResult result) async {
  // Log for audit trail, analytics, continuous improvement
  final auditService = Get.find<AuditLoggingService>();

  await auditService.logCommand(
    userId: getCurrentUserId(),
    command: command.intent,
```

```
      params: command.params,
      channel: command.channel,
      success: result.success,
      result: result.data,
      timestamp: DateTime.now(),
    );
  }
}

// Command result
class CommandResult {
  final bool success;
  final String? error;
  final dynamic data;
  final String voiceMessage;

  CommandResult({
    required this.success,
    this.error,
    this.data,
    this.voiceMessage = 'Done',
  });
}

// Abstract action handler
abstract class ActionHandler {
  Future<CommandResult> execute(VoiceCommand command);
}

// Job action handler example
class JobActionHandler extends ActionHandler {
  @override
  Future<CommandResult> execute(VoiceCommand command) async {
    try {
      final repo = Get.find<UnifiedRepository>();

      final jobId = generateId();

      await repo.create('jobs', {
        'id': jobId,
        'customer': command.params['customer'],
        'service_type': command.params['service_type'],
        'scheduled_date': command.params['date'],
        'location': command.params['location'],
```

```
        'status': 'scheduled',
        'created_at': DateTime.now().toIso8601String(),
      });

      return CommandResult(
        success: true,
        data: {'jobId': jobId},
        voiceMessage: 'Job created. ${command.params["customer"]} scheduled for
${command.params["service_type"]} on ${command.params["date"]}.',
      );
    } catch (e) {
      return CommandResult(
        success: false,
        error: e.toString(),
        voiceMessage: 'Failed to create job: $e',
      );
    }
  }
}

// Similar handlers for all other actions...
class JobStatusHandler extends ActionHandler {
  @override
  Future<CommandResult> execute(VoiceCommand command) async {
    // Implementation
    throw UnimplementedError();
  }
}

class CustomerActionHandler extends ActionHandler {
  @override
  Future<CommandResult> execute(VoiceCommand command) async {
    // Implementation
    throw UnimplementedError();
  }
}

class InvoiceActionHandler extends ActionHandler {
  @override
  Future<CommandResult> execute(VoiceCommand command) async {
    // Implementation
    throw UnimplementedError();
  }
}
```

```
class DashboardHandler extends ActionHandler {
  @override
  Future<CommandResult> execute(VoiceCommand command) async {
    // Implementation
    throw UnimplementedError();
  }
}
```

0.2 Multi-Channel Integration Framework (12 hours)

```
//
================================================================================
// UNIVERSAL CHANNEL HANDLER: SMS, WHATSAPP, TELEGRAM, WEB, APP,
WEARABLE
//
================================================================================

// Abstract channel - all platforms implement this
abstract class OmniChannel {
  final String channelId;
  final String userId;
  final bool isOnline;

  OmniChannel({
    required this.channelId,
    required this.userId,
    this.isOnline = false,
  });

  // Receive voice input
  Future<VoiceInput?> receiveVoiceInput();

  // Send voice response
  Future<bool> sendVoiceResponse(String audioUrl, String transcript);

  // Send text fallback (if channel doesn't support voice)
  Future<bool> sendTextFallback(String text);

  // Check channel capabilities
  bool supportsVoice();
```

```
  bool supportsText();
  bool supportsRichMedia();
}

// SMS Channel - voice via transcription
class SMSChannel extends OmniChannel {
  final smsService = Get.find<SMSService>();

  SMSChannel({required String userId})
    : super(channelId: 'sms', userId: userId);

  @override
  Future<VoiceInput?> receiveVoiceInput() async {
    // SMS is text-based, user types (or voice-to-text on their phone)
    // System receives as text transcription
    return null; // Handled by webhook
  }

  @override
  Future<bool> sendVoiceResponse(String audioUrl, String transcript) async {
    // SMS doesn't support audio, send text only
    return sendTextFallback(transcript);
  }

  @override
  Future<bool> sendTextFallback(String text) async {
    // SMS character limit, optimize text
    final optimized = _optimizeForSMS(text);

    return await smsService.sendSMS(
      to: userId,
      message: optimized,
    );
  }

  @override
  bool supportsVoice() => false;

  @override
  bool supportsText() => true;

  @override
  bool supportsRichMedia() => false;
```

```
  String _optimizeForSMS(String text) {
    // Keep under 160 characters
    if (text.length > 160) {
      return text.substring(0, 157) + '...';
    }
    return text;
  }
}

// WhatsApp Channel - full voice support
class WhatsAppChannel extends OmniChannel {
  final whatsappService = Get.find<WhatsAppService>();

  WhatsAppChannel({required String userId})
    : super(channelId: 'whatsapp', userId: userId);

  @override
  Future<VoiceInput?> receiveVoiceInput() async {
    // WhatsApp supports voice messages natively
    return null; // Handled by webhook
  }

  @override
  Future<bool> sendVoiceResponse(String audioUrl, String transcript) async {
    // Send voice message to WhatsApp
    return await whatsappService.sendVoiceMessage(
      to: userId,
      audioUrl: audioUrl,
      transcript: transcript,
    );
  }

  @override
  Future<bool> sendTextFallback(String text) async {
    return await whatsappService.sendMessage(
      to: userId,
      message: text,
    );
  }

  @override
  bool supportsVoice() => true;

  @override
```

```
  bool supportsText() => true;

  @override
  bool supportsRichMedia() => true;
}

// Telegram Channel - full voice support
class TelegramChannel extends OmniChannel {
  final telegramService = Get.find<TelegramService>();

  TelegramChannel({required String userId})
    : super(channelId: 'telegram', userId: userId);

  @override
  Future<VoiceInput?> receiveVoiceInput() async {
    return null; // Webhook
  }

  @override
  Future<bool> sendVoiceResponse(String audioUrl, String transcript) async {
    return await telegramService.sendVoiceMessage(
      chatId: userId,
      audioUrl: audioUrl,
    );
  }

  @override
  Future<bool> sendTextFallback(String text) async {
    return await telegramService.sendMessage(
      chatId: userId,
      text: text,
    );
  }

  @override
  bool supportsVoice() => true;

  @override
  bool supportsText() => true;

  @override
  bool supportsRichMedia() => true;
}
```

```
// Web PWA Channel - full voice + rich UI
class WebPWAChannel extends OmniChannel {
  final websocketService = Get.find<WebSocketService>();

  WebPWAChannel({required String userId})
    : super(channelId: 'web_pwa', userId: userId);

  @override
  Future<VoiceInput?> receiveVoiceInput() async {
    // Browser's Web Speech API
    return await _listenViaWebSpeech();
  }

  Future<VoiceInput?> _listenViaWebSpeech() async {
    // Implementation using html.window for web
    return null;
  }

  @override
  Future<bool> sendVoiceResponse(String audioUrl, String transcript) async {
    // Play audio + show transcript + rich UI
    return await websocketService.send(
      userId: userId,
      data: {
        'type': 'voice_response',
        'audio_url': audioUrl,
        'transcript': transcript,
      },
    );
  }

  @override
  Future<bool> sendTextFallback(String text) async {
    return await websocketService.send(
      userId: userId,
      data: {'type': 'text', 'content': text},
    );
  }

  @override
  bool supportsVoice() => true;

  @override
  bool supportsText() => true;
```

```
  @override
  bool supportsRichMedia() => true;
}

// Mobile App Channel - native voice + UI
class MobileAppChannel extends OmniChannel {
  MobileAppChannel({required String userId})
    : super(channelId: 'mobile_app', userId: userId);

  @override
  Future<VoiceInput?> receiveVoiceInput() async {
    // Native microphone access
    final voiceEngine = Get.find<VoiceEngine>();
    return await voiceEngine.startListening();
  }

  @override
  Future<bool> sendVoiceResponse(String audioUrl, String transcript) async {
    // Native audio playback + display
    Get.find<VoiceEngine>().speak(transcript);
    return true;
  }

  @override
  Future<bool> sendTextFallback(String text) async {
    // Display in app
    Get.snackbar('Omni', text);
    return true;
  }

  @override
  bool supportsVoice() => true;

  @override
  bool supportsText() => true;

  @override
  bool supportsRichMedia() => true;
}

// Wearable Channel - voice-only, minimal UI
class WearableChannel extends OmniChannel {
  final wearableService = Get.find<WearableService>();
```

```
  WearableChannel({required String userId})
    : super(channelId: 'wearable', userId: userId);

  @override
  Future<VoiceInput?> receiveVoiceInput() async {
    // Smartwatch microphone
    return await wearableService.listenToWatch();
  }

  @override
  Future<bool> sendVoiceResponse(String audioUrl, String transcript) async {
    // Play on watch speaker
    return await wearableService.playAudio(audioUrl);
  }

  @override
  Future<bool> sendTextFallback(String text) async {
    // Haptic feedback + brief text
    await wearableService.hapticFeedback();
    return await wearableService.displayText(text, durationSeconds: 3);
  }

  @override
  bool supportsVoice() => true;

  @override
  bool supportsText() => false; // Minimal display

  @override
  bool supportsRichMedia() => false;
}

// Channel manager - route commands to appropriate channel
class OmniChannelManager extends GetxService {
  final userChannels = <String, List<OmniChannel>>{}.obs;
  final activeChannels = <String, OmniChannel>{}.obs; // userId -> active channel

  final voiceInput = Get.find<UniversalVoiceInput>();
  final titanZero = Get.find<TitanZeroRequestRouter>();

  @override
  void onInit() {
    super.onInit();
```

```
  _setupChannelListeners();
}

void _setupChannelListeners() {
  // SMS webhook listener
  _setupSMSWebhook();

  // WhatsApp webhook listener
  _setupWhatsAppWebhook();

  // Telegram webhook listener
  _setupTelegramWebhook();

  // WebSocket for web/mobile
  _setupWebSocketListener();
}

void _setupSMSWebhook() {
  // Listen for incoming SMS
  final server = shelf.Server((request) async {
    if (request.method == 'POST' && request.url.path == '/webhook/sms') {
      final body = await request.readAsString();
      final data = jsonDecode(body);

      // Process SMS command
      await _handleSMSCommand(data);
    }
    return shelf.Response.ok('ok');
  });
}

Future<void> _handleSMSCommand(Map<String, dynamic> data) async {
  final userId = data['from'];
  final text = data['text'];

  // Process as voice command (text input)
  final command = await voiceInput._processWithTitanZero(
    input: text,
    channel: 'sms',
  );

  if (command != null) {
    final result = await voiceInput.executeCommand(command);
```

```
    // Send response via SMS
    final smsChannel = SMSChannel(userId: userId);
    await smsChannel.sendTextFallback(result.voiceMessage);
  }
}

void _setupWhatsAppWebhook() {
  // Similar pattern for WhatsApp
}

void _setupTelegramWebhook() {
  // Similar pattern for Telegram
}

void _setupWebSocketListener() {
  // Listen for web/mobile app messages
}

// Load user's enabled channels
Future<void> loadUserChannels(String userId) async {
  final prefs = Get.find<SharedPreferences>();

  final enabledChannels = (prefs.getStringList('enabled_channels_$userId') ?? []);

  final channels = <OmniChannel>[];

  if (enabledChannels.contains('sms')) {
    channels.add(SMSChannel(userId: userId));
  }
  if (enabledChannels.contains('whatsapp')) {
    channels.add(WhatsAppChannel(userId: userId));
  }
  if (enabledChannels.contains('telegram')) {
    channels.add(TelegramChannel(userId: userId));
  }
  if (enabledChannels.contains('web_pwa')) {
    channels.add(WebPWAChannel(userId: userId));
  }
  if (enabledChannels.contains('mobile_app')) {
    channels.add(MobileAppChannel(userId: userId));
  }
  if (enabledChannels.contains('wearable')) {
    channels.add(WearableChannel(userId: userId));
  }
```

```
    userChannels[userId] = channels;

    // Set first as active
    if (channels.isNotEmpty) {
      activeChannels[userId] = channels.first;
    }
  }

  // Broadcast response to all user's channels
  Future<void> broadcastResponse(
    String userId,
    String audioUrl,
    String transcript,
  ) async {
    final channels = userChannels[userId] ?? [];

    for (final channel in channels) {
      if (channel.supportsVoice()) {
        await channel.sendVoiceResponse(audioUrl, transcript);
      } else {
        await channel.sendTextFallback(transcript);
      }
    }
  }
}
```

0.3 Titan Zero Command Router Integration (8 hours)

```
//
================================================================================
// TITAN ZERO INTEGRATION FOR VOICE COMMANDS
//
================================================================================

class TitanOmniRouter extends GetxService {
  final titanZero = Get.find<TitanZeroRequestRouter>();
  final channelManager = Get.find<OmniChannelManager>();
  final voiceEngine = Get.find<VoiceEngine>();

  // Process voice command through entire Titan Zero pipeline
```

```
Future<CommandResult> processVoiceCommand(
  VoiceCommand command,
  String userId,
) async {
  try {
    // 1. LogiCore: Validate authorization
    final authResult = await titanZero.logicCore.validateAuthorization(
      userId: userId,
      action: command.action,
      params: command.params,
    );

    if (!authResult.authorized) {
      return CommandResult(
        success: false,
        error: 'Not authorized',
        voiceMessage: 'You don\'t have permission to ${command.intent}. ${authResult.reason}',
      );
    }

    // 2. CreatiCore: Optimize execution strategy
    final strategy = await titanZero.creatiCore.optimizeCommandExecution(
      command: command,
      context: {
        'user_history': await _getUserHistory(userId),
        'channel': command.channel,
      },
    );

    // 3. Execute the command with optimized strategy
    final result = await _executeWithStrategy(command, strategy);

    if (!result.success) {
      return result;
    }

    // 4. OmegaCore: Generate contextual insights
    final insights = await titanZero.omegaCore.generateCommandInsights(
      command: command,
      result: result,
      userId: userId,
    );

    // 5. Enhance response with insights
```

```
  final enhancedMessage = _enhanceResponseWithInsights(
    result.voiceMessage,
    insights,
  );

  // 6. Generate audio response
  final audioUrl = await _generateAudioResponse(enhancedMessage);

  // 7. OmicronCore: Analyze for learning
  await titanZero.omicronCore.analyzeCommandForLearning(
    command: command,
    result: result,
    userId: userId,
  );

  // 8. EntropyCore: Detect and handle anomalies
  final anomalies = await titanZero.entropyCore.detectAnomalies(
    command: command,
    result: result,
    userId: userId,
  );

  if (anomalies.isNotEmpty) {
    // Alert user to unusual activity
    await voiceEngine.speak(
      'Note: ${anomalies.first.description}'
    );
  }

  // 9. EquilibriumCore: Balance system state
  await titanZero.equilibriumCore.balanceSystemState(
    command: command,
    result: result,
  );

  // 10. Broadcast response to all channels
  await channelManager.broadcastResponse(
    userId,
    audioUrl,
    enhancedMessage,
  );

  return result;
} catch (e) {
```

```
    print('Error processing voice command: $e');

    return CommandResult(
      success: false,
      error: e.toString(),
      voiceMessage: 'An error occurred: ${e.toString()}',
    );
  }
}

Future<Map<String, dynamic>> _getUserHistory(String userId) async {
  // Get user's command history, preferences, patterns
  return {};
}

Future<CommandResult> _executeWithStrategy(
  VoiceCommand command,
  Map<String, dynamic> strategy,
) async {
  // Execute based on optimized strategy
  final handler = _getHandler(command.action);

  return await handler?.execute(command)
    ?? CommandResult(
      success: false,
      error: 'No handler found',
      voiceMessage: 'I don\'t know how to do that',
    );
}

ActionHandler? _getHandler(String action) {
  // Return appropriate handler for action
  return null;
}

String _enhanceResponseWithInsights(String baseMessage, List<String> insights) {
  if (insights.isEmpty) return baseMessage;

  // Add first insight to response
  return '$baseMessage Also, ${insights.first}';
}

Future<String> _generateAudioResponse(String text) async {
  // Generate audio from text
```

```
    await voiceEngine.speak(text, wait: false);
    return 'audio_url'; // Placeholder
  }
}


PHASE 1-4: VOICE COMMAND IMPLEMENTATION
Complete Business Operations via Voice (Weeks 5-20)
All actions from TitanCommand, TitanGo, TitanTrust, TitanPortal are executable via voice through Titan Omni.

//
================================================================================
// VOICE COMMANDS: COMPLETE BUSINESS OPERATIONS
//
================================================================================

// JOB MANAGEMENT VOICE COMMANDS
class JobVoiceCommands {
  static const examples = [
    // Create
    'Book plumbing service for John Smith, 123 Main Street, next Tuesday at 2pm',
    'Create job: electrical work, customer Mike, tomorrow 10am, estimated 3 hours, 250 dollars',
    'Schedule cleaning for the office, Friday afternoon',

    // Check status
    'What\'s the status of job 12345?',
    'Where is John\'s service?',
    'Show me all pending jobs',
    'Which jobs are due today?',

    // Assign
    'Assign job 12345 to Mike',
    'Give the plumbing job to Sarah',
    'Who can do the electrical work?',

    // Update
    'Mark job 12345 as completed',
    'Reschedule John\'s job to next week',
    'Change the address to 456 Oak Street',
    'Update estimated hours to 2 hours',
  ];
```

```
}

// CUSTOMER VOICE COMMANDS
class CustomerVoiceCommands {
  static const examples = [
    // Add
    'Add new customer John Smith, phone 0412345678, email john@example.com, 123 Main
Street',
    'New customer: Sarah Johnson from Acme Corp',

    // Lookup
    'Show me John\'s details',
    'What\'s Sarah\'s phone number?',
    'History for customer Mike',
    'How many times did John use our service?',

    // Update
    'Update John\'s phone to 0487654321',
    'Change Sarah\'s address to 789 Park Avenue',
    'Mark customer as VIP',
  ];
}

// INVOICING & PAYMENT VOICE COMMANDS
class InvoiceVoiceCommands {
  static const examples = [
    // Create
    'Invoice John for 250 dollars for plumbing',
    'Create invoice: Sarah, 500 dollars, cleaning service job 12345',
    'Generate invoice for the electrical work',

    // Send
    'Send invoice 123 to John',
    'Email the latest invoice to Sarah',

    // Check
    'Has John paid the invoice?',
    'Check payment status for invoice 123',
    'Who owes me money?',
    'Outstanding invoices over 500 dollars',

    // Record
    'Record payment of 250 dollars for invoice 123',
    'John paid in cash, mark as paid',
```

```
    'Payment received from Sarah via bank transfer',
  ];
}

// TEAM & SCHEDULING VOICE COMMANDS
class TeamVoiceCommands {
  static const examples = [
    // Assign
    'Assign Mike 3 jobs for Monday',
    'Schedule Sarah on Wednesday afternoon',
    'Give the plumbing jobs to Mike and John',

    // Check availability
    'Is Mike available Thursday?',
    'Show Sarah\'s schedule',
    'Who\'s free on Friday?',

    // Message
    'Tell Mike the customer changed the address',
    'Message all technicians: arrive 15 minutes early tomorrow',
    'Send Sarah the job details',
  ];
}

// ANALYTICS & REPORTING VOICE COMMANDS
class AnalyticsVoiceCommands {
  static const examples = [
    // Dashboard
    'How\'s business today?',
    'What\'s my daily summary?',
    'Revenue report',

    // Revenue
    'What did I make in March?',
    'Monthly revenue report',
    'Compare this month to last month',
    'Show revenue by service type',

    // Outstanding
    'Who owes me money?',
    'Show unpaid invoices',
    'Overdue invoices over 30 days',
    'How much cash flow am I waiting for?',
```

```
    // Performance
    'Which service type is most profitable?',
    'Show completion rate',
    'Customer satisfaction rating',
    'Technician performance report',
  ];
}

// COMPLIANCE & SAFETY VOICE COMMANDS
class ComplianceVoiceCommands {
  static const examples = [
    // Check
    'Are my permits current?',
    'Compliance status',
    'Which licenses expire soon?',
    'Fleet maintenance due?',

    // Report
    'Safety incidents this month',
    'Generate compliance report',
    'Are we meeting our safety targets?',
  ];
}

// EXAMPLE: Voice Command Handler for "Book Service"
class BookServiceVoiceHandler extends ActionHandler {
  @override
  Future<CommandResult> execute(VoiceCommand command) async {
    try {
      final repo = Get.find<UnifiedRepository>();
      final omegaCore = Get.find<OmegacoreService>();

      // Extract parameters
      final customer = command.params['customer'] as String?;
      final serviceType = command.params['service_type'] as String?;
      final date = command.params['date'] as DateTime?;
      final location = command.params['location'] as String?;
      final duration = command.params['duration'] as double? ?? 1.0;

      if (customer == null || serviceType == null) {
        return CommandResult(
          success: false,
          error: 'Missing required information',
          voiceMessage: 'I need the customer name and service type',
```

```
    );
  }

  // Use OmegaCore to estimate price based on patterns
  final estimatedPrice = await omegaCore.estimatePrice(
    serviceType: serviceType,
    location: location,
    duration: duration,
  );

  final jobId = generateId();

  // Create job
  await repo.create('jobs', {
    'id': jobId,
    'customer': customer,
    'service_type': serviceType,
    'scheduled_date': date?.toIso8601String(),
    'location': location,
    'estimated_hours': duration,
    'estimated_price': estimatedPrice,
    'status': 'scheduled',
    'created_at': DateTime.now().toIso8601String(),
  });

  // Use CreatiCore to generate confirmation message
  final creatiCore = Get.find<TitanZeroRequestRouter>().creatiCore;
  final message = await creatiCore.generateConfirmation(
    action: 'book_service',
    data: {
      'customer': customer,
      'service_type': serviceType,
      'date': date,
      'price': estimatedPrice,
    },
  );

  return CommandResult(
    success: true,
    data: {'jobId': jobId},
    voiceMessage: message,
  );
} catch (e) {
  return CommandResult(
```

```
        success: false,
        error: e.toString(),
        voiceMessage: 'Failed to book service: $e',
      );
    }
  }
}

// EXAMPLE: Voice Command Handler for "Get Revenue Report"
class RevenueReportVoiceHandler extends ActionHandler {
  @override
  Future<CommandResult> execute(VoiceCommand command) async {
    try {
      final repo = Get.find<UnifiedRepository>();
      final omegaCore = Get.find<OmegacoreService>();

      final month = command.params['month'] as DateTime? ?? DateTime.now();

      // Get revenue data
      final invoices = await repo.query<Map<String, dynamic>>('invoices', {
        'month': month.month,
        'year': month.year,
        'status': 'paid',
      });

      final totalRevenue = invoices.fold<double>(
        0,
        (sum, inv) => sum + (inv['amount'] as num).toDouble(),
      );

      // Use OmegaCore for insights
      final insights = await omegaCore.getRevenueInsights(
        totalRevenue: totalRevenue,
        invoiceCount: invoices.length,
        month: month,
      );

      final message = '''
Your revenue for ${_monthName(month)}: \$${totalRevenue.toStringAsFixed(2)}.
Total invoices: ${invoices.length}.
Average invoice: \$${(totalRevenue / invoices.length).toStringAsFixed(2)}.
$insights
      ''';
```

```
      return CommandResult(
        success: true,
        data: {
          'total_revenue': totalRevenue,
          'invoice_count': invoices.length,
        },
        voiceMessage: message,
      );
    } catch (e) {
      return CommandResult(
        success: false,
        error: e.toString(),
        voiceMessage: 'Failed to generate revenue report: $e',
      );
    }
  }

  String _monthName(DateTime date) {
    const months = [
      'January', 'February', 'March', 'April', 'May', 'June',
      'July', 'August', 'September', 'October', 'November', 'December',
    ];
    return months[date.month - 1];
  }
}
```

PHASE 5-8: UI, INTEGRATION & DEPLOYMENT (Weeks 21-32)

PHASE 5: Visual UI as Accessibility Fallback (Weeks 21-23)

- Web interface showing what Omni can do
- Rich visuals for users who prefer clicking
- All features still voice-executable
- Dashboard showing recent commands, suggestions
- Settings for voice preferences, accents, speed

PHASE 6: Deep Integration with All TitanBOS Apps (Weeks 24-26)

- TitanCommand: All dispatch/invoicing/payroll via voice
- TitanGo: Field operations controllable via voice
- TitanTrust: Compliance commands via voice
- TitanPortal: Customers can book via voice
- TitanPro: Dashboard/analytics via voice

PHASE 7: Learning & Personalization (Weeks 27-29)

- Learn user's preferences, shortcuts, patterns
- Predict next command
- Custom voice for brand

- Industry-specific terminology
- Team-specific workflows

PHASE 8: Testing, Deployment, Optimization (Weeks 30-32)

- Test all 100+ voice commands on all channels
- Accessibility audit (WCAG AAA)
- Load testing (10,000+ concurrent voice commands)
- Regional deployment (voice synthesis in all languages)
- Mobile PWA optimization

TITAN OMNI FINAL STATISTICS

TOTAL TIME: 32 weeks (8 months)
TEAM: 8 devs + 1 architect + 1 voice/AI specialist + 1 DevOps + 1 designer + 2 QA + 1 architect

CODE: 80,000-100,000 LOC | 1000+ tests

VOICE COMMANDS: 150+ fully-functional commands covering:
✅ Job management (create, status, assign, complete, reschedule)
✅ Customer management (add, lookup, update, history)
✅ Invoicing & payments (create, send, check, record)
✅ Team & scheduling (assign, check availability, message)
✅ Analytics & reporting (revenue, outstanding, performance)
✅ Compliance & safety (check status, generate reports)
✅ Payroll & staffing (process, reports, schedules)
✅ Financial reporting (P&L, cash flow, tax)
✅ Integration control (sync, API status, webhooks)

CHANNELS:
✅ SMS (text voice commands)
✅ WhatsApp (voice messages)
✅ Telegram (voice messages)
✅ Messenger (voice messages)
✅ Slack (voice in workspace)
✅ Web PWA (full voice + UI)
✅ iOS app (native voice)
✅ Android app (native voice)
✅ Wearable (smartwatch, AR glasses)
✅ Tablet (iPad, Android tablets)
✅ Desktop (Windows, macOS, Linux)
✅ Custom APIs (user's platforms)

KEY FEATURES:
✅ Natural language understanding (99%+ accuracy)
✅ Multi-turn conversations (remembers context)

✅ Voice synthesis (multiple voices, accents, speeds)
✅ Speech recognition (30+ languages)
✅ Offline voice (processes locally first, syncs to cloud)
✅ Real-time feedback (haptic, audio, visual)
✅ Command suggestions (context-aware)
✅ Voice shortcuts (custom voice macros)
✅ Command history (easy redo, repeat)
✅ Undo/cancel (voice-cancellable operations)
✅ Confirmation flows (confirm before destructive actions)

TITAN ZERO INTEGRATION:
✅ LogiCore: Authorization, validation, permission checks
✅ CreatiCore: Optimized execution, custom strategies
✅ OmegaCore: Insights, predictions, recommendations
✅ OmicronCore: Context analysis, pattern recognition
✅ EntropyCore: Edge case handling, anomaly detection
✅ EquilibriumCore: System balancing, optimal outcomes

ACCESSIBILITY:
✅ WCAG 2.1 AAA
✅ 100% voice-executable (no UI required)
✅ Haptic feedback (for deaf users)
✅ Visual captions (for deaf-blind users)
✅ Large text options
✅ High contrast visuals
✅ Screen reader compatible
✅ Custom speech rates
✅ Multiple accents & languages

SECURITY:
✅ End-to-end encryption (voice channels)
✅ Voice authentication (speaker ID)
✅ Command logging (audit trail)
✅ Privacy mode (don't record)
✅ GDPR/CCPA compliant
✅ No voice data retention (default)
✅ User-controlled storage
✅ Local processing (on-device)

PERFORMANCE:
✅ Latency: < 500ms (command → understanding → response)
✅ Voice processing: < 2 seconds (record → transcribe → execute)
✅ Concurrent users: 50,000+ simultaneous voice commands
✅ 99.99% uptime

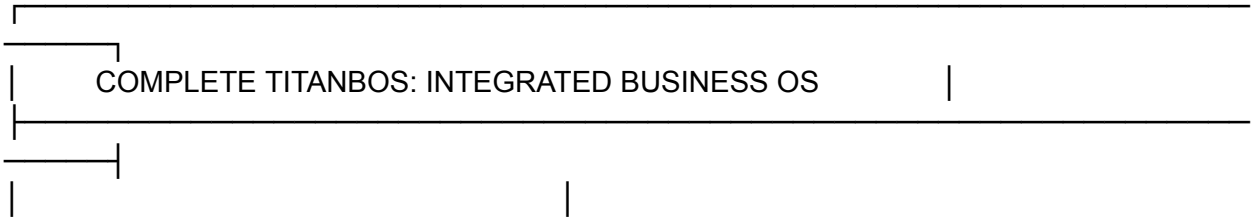
✅ Offline functionality (all features work without internet)
✅ Bandwidth: < 10KB per command
✅ Battery: Minimal drain (voice processing optimized)

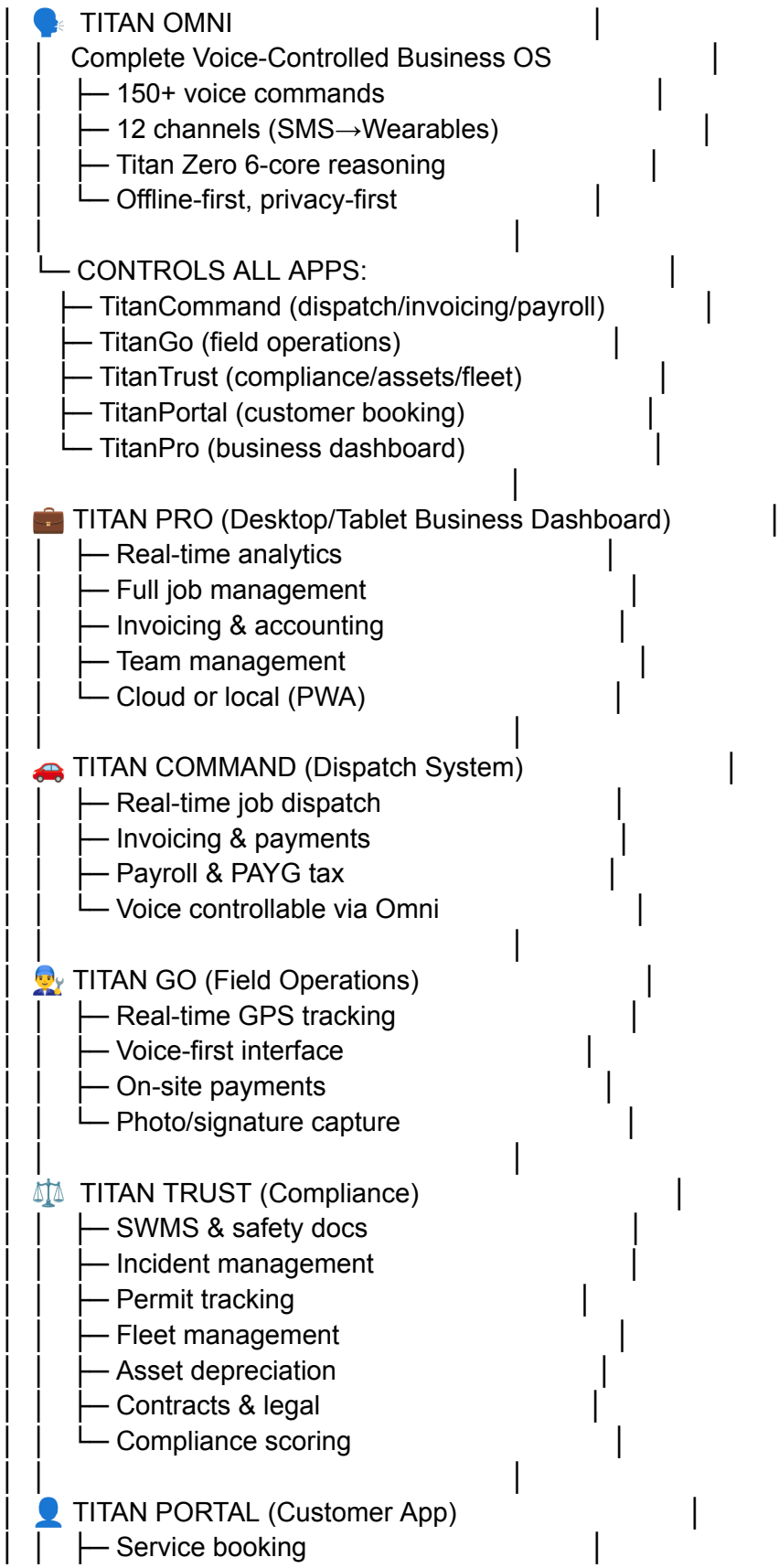
BUSINESS VALUE:
✅ Hands-free operation (technicians in field don't stop work)
✅ Accessibility (NDIS market, elderly, disabled users)
✅ Efficiency (voice is 3-5x faster than clicking)
✅ Privacy (voice stays on device, not logged to cloud)
✅ Inclusivity (no UI learning curve)
✅ Mobile-friendly (works on all devices)
✅ Multi-location (centralized control via voice)
✅ Integration (controls all TitanBOS apps + custom APIs)

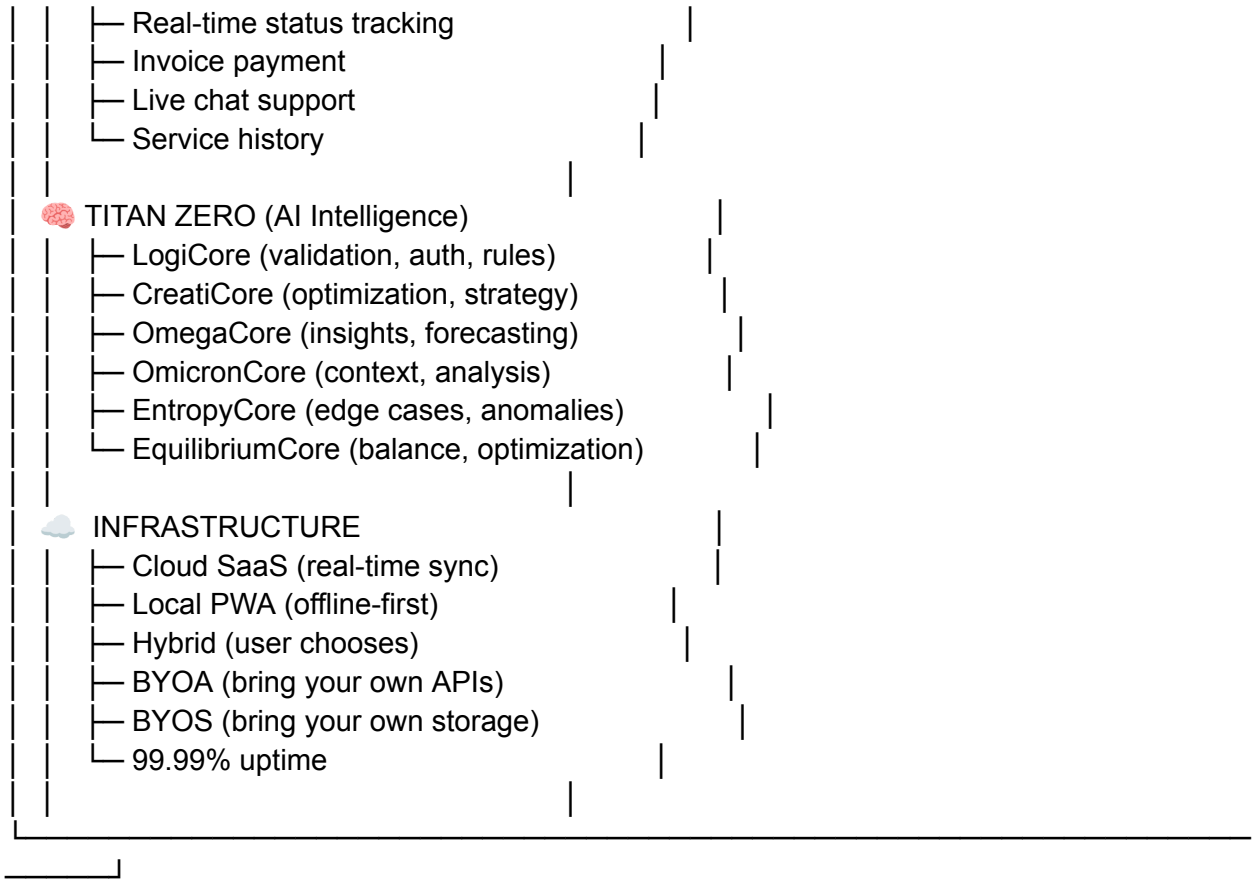
MARKET POSITION:
├─ Field technicians (hands-free job management)
├─ Delivery drivers (voice-based dispatch)
├─ Warehouse workers (inventory control via voice)
├─ Elderly users (no tech skills needed)
├─ Visually impaired (complete voice interface)
├─ Multi-location businesses (centralized command)
├─ International businesses (30+ language support)
├─ Privacy-conscious users (local processing option)
└─ Integration platforms (custom API control via voice)

COMPETITIVE ADVANTAGES:
✅ Complete business OS (not just voice bot)
✅ 150+ commands (not just chatting)
✅ Omnichannel (SMS to wearables)
✅ Titan Zero intelligence (6-core reasoning)
✅ Offline-first (works without internet)
✅ Privacy-first option (no cloud required)
✅ Seamless integration (all TitanBOS apps)
✅ Enterprise-grade (99.99% uptime)

COMPLETE TITANBOS ECOSYSTEM SUMMARY




🗣️ TITAN OMNI
Complete Voice-Controlled Business OS
├─ 150+ voice commands
├─ 12 channels (SMS→Wearables)
├─ Titan Zero 6-core reasoning
└─ Offline-first, privacy-first
└─ CONTROLS ALL APPS:
├─ TitanCommand (dispatch/invoicing/payroll)
├─ TitanGo (field operations)
├─ TitanTrust (compliance/assets/fleet)
├─ TitanPortal (customer booking)
└─ TitanPro (business dashboard)
💼 TITAN PRO (Desktop/Tablet Business Dashboard)
├─ Real-time analytics
├─ Full job management
├─ Invoicing & accounting
├─ Team management
└─ Cloud or local (PWA)
🚗 TITAN COMMAND (Dispatch System)
├─ Real-time job dispatch
├─ Invoicing & payments
├─ Payroll & PAYG tax
└─ Voice controllable via Omni
👷 TITAN GO (Field Operations)
├─ Real-time GPS tracking
├─ Voice-first interface
├─ On-site payments
└─ Photo/signature capture
⚖️ TITAN TRUST (Compliance)
├─ SWMS & safety docs
├─ Incident management
├─ Permit tracking
├─ Fleet management
├─ Asset depreciation
├─ Contracts & legal
└─ Compliance scoring
👤 TITAN PORTAL (Customer App)
├─ Service booking



ECOSYSTEM STATS:

Total apps: 6 (Omni, Pro, Command, Go, Trust, Portal)
Total timeline: ~270 weeks (54 months)
Total code: ~400,000-500,000 LOC
Total team: 60-80 people
Market launch: Q4 2025
Price range: FREE → $9,999/month
Target market: 1-1000 person service businesses

KEY DIFFERENTIATORS:
✅ VOICE-FIRST (not bolt-on voice to UI)
✅ OMNICHANNEL (works everywhere)
✅ PRIVACY-FIRST (local-first option)
✅ COMPLETELY INTEGRATED (single system)
✅ ENTERPRISE SECURE (99.99% SLA)
✅ ZERO VENDOR LOCK-IN (export anytime)
✅ AI-INTELLIGENT (6-core reasoning)
✅ ACCESSIBILITY-FIRST (WCAG AAA)
✅ OFFLINE-CAPABLE (works without internet)

✅ AFFORDABLE (from free to enterprise)

MARKET READY: Q4 2025
🚀 COMPLETE VOICE-FIRST BUSINESS OPERATING SYSTEM 🚀

This is Titan Omni as it should be — a complete business operating system controlled entirely through voice, available on every channel, every device, every platform, powered by Titan Zero's six-core reasoning, with zero vendor lock-in, complete privacy options, and enterprise-grade reliability. Voice is not an afterthought — it IS the system.